

Xenmesh Fashion House - Admin Authentication

Implementation summary | Commit b56af07 | October 8, 2026

Overview

Added a NestJS authentication backend for the admin panel using email and password, Prisma 7, MySQL, and secure browser sessions. Authentication is required for API routes unless a route is explicitly public.

Backend changes

- Global Nest authentication guard protects routes by default; POST /auth/sign-in is explicitly public.
- POST /auth/sign-in validates credentials, checks that the account is an ADMIN, verifies the password hash, and creates a signed-in session. Invalid credentials receive a generic error.
- GET /auth/me returns the currently authenticated admin. POST /auth/sign-out ends the current session.
- The authentication package uses an HttpOnly session cookie, so browser JavaScript cannot read the session token.
- Sessions are stored in MySQL in the new AuthSession table and are linked to User records.
- Password hashing uses Nest authentication package bcrypt hashing; older hashes are upgraded after successful sign-in.
- CORS is configured for the local frontend and the backend defaults to port 3001.

Admin account seed

Run `npm run seed:admin` from the backend folder. The command builds the backend, loads `backend/.env`, and ensures an administrator exists.

- Set `ADMIN_EMAIL` and `ADMIN_PASSWORD` in the actual `backend/.env` file; `.env.example` is only a template.
- Use a password of at least 12 characters. The seed rejects missing settings or invalid values rather than using a built-in password.
- If the email is new, the seed creates the account. If it belongs to an existing customer, the account is promoted without replacing its password. It is safe to rerun.
- The backend also ensures this configured admin account at startup.

Database changes

- New users default to the ADMIN role in the Prisma schema.
- A migration creates AuthSession with session expiry, activity, optional MFA state, metadata, and a cascading User relation.
- The migration was applied to the local MySQL database and Prisma reported the schema up to date.

Setup, verification, and scope

Local setup

1. Keep existing database settings in backend/.env. Add the admin and local app settings below, using your own password:

```
ADMIN_EMAIL=admin@xenmesh.local
```

```
ADMIN_PASSWORD=your-own-password-at-least-12-characters
```

```
PORT=3001
```

```
FRONTEND_URL=http://localhost:3000
```

2. From backend, run `npm run seed:admin`, then `npm run start:dev`.

3. Set `NEXT_PUBLIC_API_URL=http://localhost:3001` in frontend/.env.local, start the frontend, and sign in at `http://localhost:3000/login` using the same admin credentials.

Verification performed

- NestJS backend build and lint passed.
- Five focused admin authentication unit tests passed, covering successful sign-in, password hash upgrade, customer rejection, unknown-email verification, and malformed credentials.
- Next.js production build and frontend lint passed.
- Prisma schema validation passed; the MySQL migration was applied and migration status was clean.
- Runtime smoke checks confirmed protected API routes return 401 without a session, while sign-in remains public and validates its request.

Commit and frontend note

Commit `b56af07` (feat: add admin authentication) contains the backend authentication module, Prisma schema and migration, seed command, focused tests, and backend setup documentation.

The frontend directory is tracked as a Git link in the parent checkout. Its login page, admin page, and route-protection changes exist in the local workspace but were not included in this parent-repository commit. Commit those frontend changes from the frontend repository if you need them versioned there.

Production follow-up

This is configured for local admin use. Before production, persist any additional authentication stores required by enabled features and add login rate limiting. Use production secrets and HTTPS settings appropriate to the deployed domain.

No actual .env credentials or passwords are included in this report.